A Course End Project Report on

Movie Recommendation DB

A8517-Database Management Systems Laboratory

Submitted by

BatchNo.22

AAFIYA TABUSAM
(24881A05D1)
BANAVATH RAJINI
(24881A05D4)
AMGOTH ARUNA
(25885A0516)

Course Facilitator

Mr. Janaki Rama Venkata Varma Bommidi
Assistant Professor, CSE

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

April 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Movie Recommendation DB"** is carried out by **"AAFIYA TABASUM", "BANAVATH RAJINI"** **and** **"AMGOTH ARUNA"** **with** Roll Numbers **"24881A05D1", "24881A05D4"** **and** **"25885A0516"** towards **A8517–Database Management Systems Laboratory** course in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Mr. Janaki Rama Venkata Varma Bommidi
Assistant Professor, CSE

Signature of the HOD
Dr. Ramesh Karnati
Associate Professor.HOD.CSE

Contents

1 Introduction 3
2 LiteratureReview 3
3 Objectives 3
4 Methodology 3
4.1 Architectural Diagram 3
4.2 Database Design(ER diagram, Entity Descriptions, Relationships and cardinality)
4.3 Relational Model(Schema Definition, Tables with primary key, foreign key with Queries)
4.4 Normalization (Justify schema is following 1NF, 2NF, 3NF. If not, explain removal of redundancy)
4.5 Hardware and Software Requirements 3
4.6 WorkingPrinciple 4
4.7 Database Implementation(DML, DDL SQL queries, ALL queries used in project,constraints , triggers, stored procedures or cursors if any used) 4
4.8 Advanced DBMS concepts (Explanation on how ACID properties, concurrency control and indexing is done)
4.9 Software Implementation (about front end and back end code with GUI)
5 Results and Discussion(GUI Screenshots, Queries outputs with or without GUI) 4
6 Mapping of POs and SDGs 5
6.1 Program Outcomes(POs) Mapping 5
6.2 Sustainable Development Goals(SDGs) Mapping 5
7 Conclusion 5
Bibliography 6

Abstract

A Movie Recommendation Database system is designed to help users discover movies based on their interests and preferences. With the rapid growth of digital content, users often face difficulty in selecting suitable movies from a vast collection. This system aims to solve this problem by organizing movie data and providing personalized recommendations.

The system stores information such as movie titles, genres, ratings, cast, and user reviews in a structured database. It analyzes user behavior, preferences, and past ratings to suggest relevant movies. Recommendation techniques such as content-based filtering and collaborative filtering can be used to improve accuracy.

Data for the system can be obtained from sources like MovieLens or APIs such as TMDb. The database ensures efficient storage, retrieval, and management of movie-related information.

This project is useful for applications like streaming platforms, entertainment websites, and mobile apps. It enhances user experience by providing quick, accurate, and personalized movie suggestions.

Keywords: Movie Recommendation System, Database Management System (DBMS), Collaborative Filtering, Content-Based Filtering, User Preferences, Movie Ratings, Data Analysis, MovieLens Dataset, TMDb API

1 Introduction

The Movie Recommendation Database system is designed to assist users in discovering movies that match their interests by analyzing large volumes of movie-related data. With the increasing availability of digital entertainment, users often find it difficult to choose relevant content from extensive catalogs. This project focuses on organizing and managing movie data using a Database Management System (DBMS) and applying recommendation techniques to provide personalized suggestions. The system typically stores information such as movie titles, genres, ratings, user preferences, and reviews, which can be obtained from datasets like MovieLens or APIs such as TMDb. Efficient data storage, retrieval, and processing play a crucial role in ensuring accurate and fast recommendations.

From a theoretical perspective, the system is based on concepts from database design, data analysis, and machine learning. Two major recommendation techniques are commonly used: content-based filtering and collaborative filtering. Content-based filtering suggests movies based on similarities between movie attributes (such as genre or keywords), often calculated using similarity measures like cosine similarity. Collaborative filtering, on the other hand, predicts user preferences by analyzing patterns in user–item interactions, such as ratings, using methods like user-based or item-based similarity. Matrix factorization techniques may also be applied to improve recommendation accuracy by decomposing large user–movie rating matrices into lower-dimensional representations. These mathematical and computational approaches enable the system to deliver personalized and relevant movie recommendations efficiently.

2 Literature Review

The development of movie recommendation systems has been widely studied in the fields of data science and machine learning. Early research primarily focused on collaborative filtering techniques, where recommendations are generated based on user–item interactions such as ratings and viewing history. Datasets like MovieLens have been extensively used in academic research to evaluate the performance of these algorithms. Studies show that user-based and item-based collaborative filtering methods can effectively predict user preferences, but they often suffer from challenges such as data sparsity and the cold-start problem.

To overcome these limitations, researchers introduced content-based filtering approaches, which rely on movie attributes such as genre, cast, and keywords. These methods use similarity measures like cosine similarity to recommend movies that are similar to those a user has previously liked. Modern systems often combine both collaborative and content-based techniques into hybrid models to improve recommendation accuracy. Existing solutions used by platforms such as Netflix and Amazon demonstrate the effectiveness of hybrid recommendation systems in real-world applications.

Recent advancements in recommendation systems include the use of machine learning and deep learning techniques such as matrix factorization, neural networks, and clustering algorithms. APIs like TMDb provide rich and dynamic datasets that support real-time recommendation systems. Despite these advancements, challenges such as scalability, diversity of recommendations, and maintaining user privacy remain active areas of research. These studies highlight the need for efficient, scalable, and accurate recommendation systems, forming the foundation for the proposed Movie Recommendation Database project.

A literature review is a comprehensive survey of scholarly sources (such as books, journal articles, prerequisites, Existing methodology or solution and reports) that relate to a specific research question or area of study. It helps to understand the current state of knowledge and identify areas for further investigation.

3.Objectives

• To design and implement a Movie Recommendation Database system that efficiently stores, manages, and retrieves movie-related data.
• To develop a recommendation mechanism using techniques such as collaborative filtering and content-based filtering to provide personalized movie suggestions.
• To integrate external data sources like MovieLens or TMDb API for enhancing the accuracy and scalability of the system.

4.Methodology

4.1 Architecture Diagram

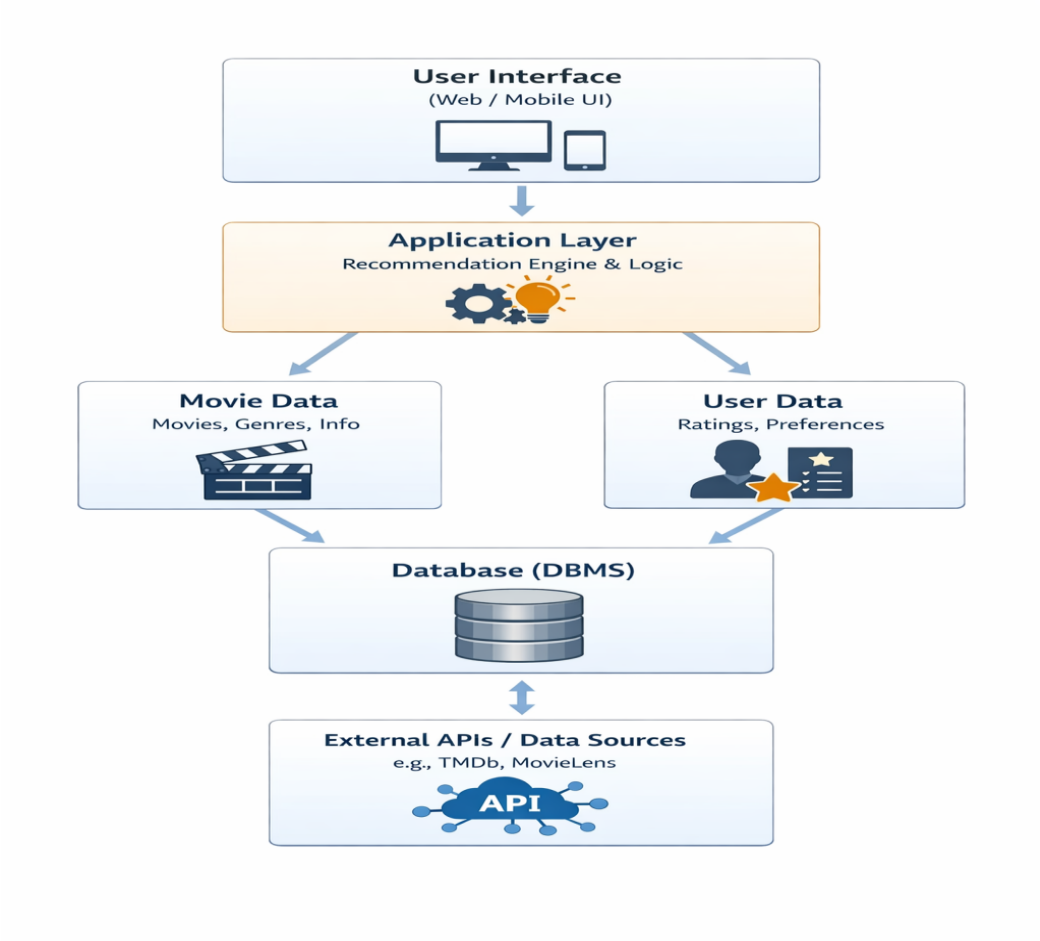
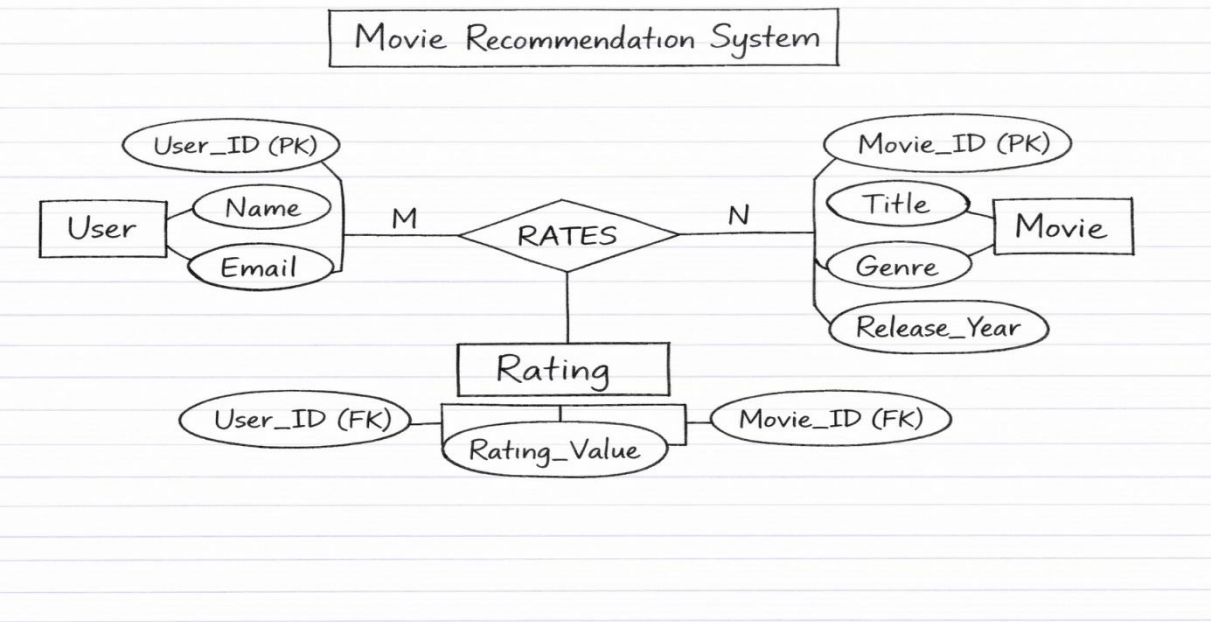


Figure 1: Architecture of Movie Recommendation System

Architecture of Movie Recommendation System – This diagram illustrates the overall structure of the system, including the User Interface, Application Layer (Recommendation Engine), Movie Data, User Data, Database (DBMS), and external data sources such as TMDb and MovieLens, showing how these components interact to generate personalized movie recommendations.

4.2 Database Design(ER diagram, Entity Descriptions, Relationships and cardinality)



1. User Entity

The User entity represents individuals who interact with the system.

Attributes:

- User_ID (Primary Key) – Unique identifier for each user
- Name – Name of the user
- Email – User's email address

Description:

Each user can browse movies and give ratings. This entity is essential because recommendations are generated based on user preferences and past ratings.

2. Movie Entity

The Movie entity stores all details related to movies available in the system.

Attributes:

- Movie_ID (Primary Key) – Unique identifier for each movie
- Title – Name of the movie
- Genre – Category (Action, Comedy, etc.)
- Release_Year – Year of release

Description:

This entity contains all movie-related information. It is used to filter and recommend movies based on genre and popularity.

3. Rating Entity (Associative Entity)

The Rating entity connects User and Movie.

Attributes:

- User_ID (Foreign Key) – References User
- Movie_ID (Foreign Key) – References Movie
- Rating_Value – Rating given by user (e.g., 1–5)

Description:

This is an associative (bridge) entity used to resolve the many-to-many relationship between User and Movie.It stores how much a user likes a movie, which is the core data used for generating recommendations.

Cardinality:

User and Movie have a many-to-many (M:N) relationship, as one user can rate many movies and one movie can be rated by many users.

This is resolved using the Rating entity, resulting in:

- User → Rating = 1:M
- Movie → Rating = 1:M

4.4 Normalization (Justify schema is following 1NF, 2NF, 3NF. If not, explain removal of redundancy)

The database schema of the Movie Recommendation System is designed following normalization principles up to the Third Normal Form (3NF) to ensure minimal redundancy and better data integrity.

In First Normal Form (1NF), all the attributes in the tables contain atomic values, and

there are no repeating groups or multi-valued attributes. Each field stores only a single value, such as one rating per record and one email per user, ensuring the table structure is simple and organized.

In Second Normal Form (2NF), the schema ensures that all non-key attributes are fully dependent on the entire primary key. In the case of the Ratings table, which uses a composite primary key (User_ID, Movie_ID), the attribute Rating_Value depends on both keys together and not on a part of the key. This removes partial dependency and improves consistency.

In Third Normal Form (3NF), the database eliminates transitive dependencies, meaning non-key attributes depend only on the primary key and not on other non-key attributes. For example, movie details such as Title and Genre depend only on Movie_ID, and user details depend only on User_ID.

Thus, the schema is fully normalized up to 3NF, which reduces data redundancy, avoids anomalies, and ensures efficient data storage and retrieval.

4.5 Hardware and Software Requirements

Hardware Requirements

- Minimum 4GB RAM
- Processor: Intel i3 or above
- Storage: 500GB

Software Requirements

- Operating System: Windows/Linux
- DBMS: MySQL / PostgreSQL
- Frontend: HTML, CSS (optional)
- Backend: Python / Java (optional)

4.6 Working Principle

The Movie Recommendation System works by analyzing user preferences and ratings stored in the database. When a user rates movies, the system records this information and identifies patterns such as preferred genres or highly rated movies.

Based on this data, the system recommends movies by selecting similar genre movies or top-rated movies using SQL queries. The recommendation process mainly involves filtering, joining tables, and sorting results to display the most relevant movies to the user.

This approach ensures efficient and personalized recommendations using database operations without requiring complex algorithms.

4.7 Database Implementation(DML, DDL SQL queries, ALL queries used in project,constraints , triggers, stored procedures or cursors if any used)

The Movie Recommendation System is implemented using a relational database with structured query language (SQL). The database is designed using Data Definition Language (DDL) to create tables such as Users, Movies, and Ratings. Each table is defined with appropriate primary keys and foreign keys to maintain referential integrity. Constraints like UNIQUE and NOT NULL are used to ensure data accuracy and consistency.

Data Manipulation Language (DML) queries are used to insert, update, delete, and retrieve data from the database. For example, user ratings are inserted into the Ratings table, and movie details are retrieved using SELECT queries for generating recommendations. The recommendation functionality is achieved using SQL queries that involve JOIN operations, filtering, and sorting based on user preferences and ratings.

Additionally, database constraints ensure valid data entry, such as restricting rating values within a specific range. Optional features like triggers can be used to validate inputs automatically, and stored procedures can be implemented to retrieve top-rated movies efficiently. Overall, the database implementation ensures efficient storage, retrieval, and management of movie and user data.

4.8 Advanced DBMS concepts (Explanation on how ACID properties, concurrency control and indexing is done)

The system follows the fundamental ACID properties to maintain reliable database transactions. Atomicity ensures that each transaction, such as inserting a user rating, is either fully completed or not executed at all. Consistency ensures that the database remains in a valid state after every transaction by enforcing constraints. Isolation allows multiple users to access and update the database simultaneously without interfering with each other. Durability guarantees that once data is committed, it is permanently stored even in case of system failures.

Concurrency control mechanisms are used to handle multiple users accessing the system at the same time. Techniques such as locking and transaction management prevent data conflicts and ensure correct updates. This is particularly important when multiple users rate movies simultaneously.

Indexing is applied to frequently accessed attributes such as Movie_ID and Genre to improve the performance of query execution. By using indexes, the system can quickly

retrieve relevant movies, making the recommendation process faster and more efficient.

4.9 Software Implementation

The Movie Recommendation System is implemented using:

- Front-End: Simple input forms (or SQL*Plus interface)
- Back-End: Oracle Database (Oracle SQL & PL/SQL)

The system stores movie data and recommends movies based on genre and rating.

Step 1:Create tables:

1. Movies Table

- Stores movie details
- Columns: movie_id, title, genre, release_year

2. Users Table

- Stores user information
- Columns: user_id, name, email

3. Ratings Table

- Stores user ratings for movies
- Columns: user_id, movie_id, rating_value

Step 2: Insert Sample Data

Table 1:

INSERT INTO movies VALUES (1, 'Baahubali', 'Action', 2015);

INSERT INTO movies VALUES (2, 'RRR', 'Action', 2022);

INSERT INTO movies VALUES (3, 'Eega', 'Fantasy', 2012);

INSERT INTO movies VALUES (4, 'Pushpa', 'Action', 2021);

INSERT INTO movies VALUES (5, 'Arjun Reddy', 'Drama', 2017);

INSERT INTO movies VALUES (6, 'Srimanthudu', 'Drama', 2015);

INSERT INTO movies VALUES (7, 'Magadheera', 'Action', 2009);

Table 2:

```
INSERT INTO users VALUES (101, 'User1', 'user1@gmail.com');
INSERT INTO users VALUES (102, 'User2', 'user2@gmail.com');
INSERT INTO users VALUES (103, 'User3', 'user3@gmail.com');
INSERT INTO users VALUES (104, 'User4', 'user4@gmail.com');
```

Table 3:

```
INSERT INTO ratings VALUES (101, 1, 5);
INSERT INTO ratings VALUES (101, 2, 4);
INSERT INTO ratings VALUES (101, 3, 5);

INSERT INTO ratings VALUES (102, 1, 4);
INSERT INTO ratings VALUES (102, 4, 5);
INSERT INTO ratings VALUES (102, 7, 5);

INSERT INTO ratings VALUES (103, 2, 5);
INSERT INTO ratings VALUES (103, 5, 4);
INSERT INTO ratings VALUES (103, 6, 4);

INSERT INTO ratings VALUES (104, 3, 5);
INSERT INTO ratings VALUES (104, 4, 4);
INSERT INTO ratings VALUES (104, 7, 5);
```

Step 3: Test Relationships (JOIN Query)

```
SELECT u.name, m.title, r.rating_value
FROM users u
JOIN ratings r ON u.user_id = r.user_id
JOIN movies m ON m.movie_id = r.movie_id;
```

Step 4: Top Rated Movies

```
SELECT m.title, AVG(r.rating_value) AS avg_rating

FROM movies m

JOIN ratings r ON m.movie_id = r.movie_id

GROUP BY m.title

ORDER BY avg_rating DESC;
```

Step 5: User-Based Recommendation

```
SELECT m.title, r.rating_value

FROM movies m

JOIN ratings r ON m.movie_id = r.movie_id

WHERE r.user_id = 101

ORDER BY r.rating_value DESC;
```

Step 6: Create VIEW

```
CREATE VIEW top_movies AS

SELECT m.title, AVG(r.rating_value) AS avg_rating

FROM movies m

JOIN ratings r ON m.movie_id = r.movie_id

GROUP BY m.title;

SELECT * FROM top_movies;
```

4 Results and Discussion

```
SQL> select *from movies;

  MOVIE_ID TITLE                                                                                    GENRE     RELEASE_YEAR
---------- ------------------------------------------------------------------------------------------ ------------------------------------------------ ---
-----
         1 Baahubali                                                                                 Action    2015
         2 RRR                                                                                       Action    2022
         3 Eega                                                                                      Fantasy    2012
         4 Pushpa                                                                                    Action    2021
         5 Arjun Reddy                                                                               Drama     2017
         6 Srimanthudu                                                                               Drama     2015
         7 Magadheera                                                                                Action    2009

7 rows selected.

SQL> select *from Users;

   USER_ID NAME                                           EMAIL
---------- ---------------------------------------------- ------------------------------------------------------------------------------------------
       101 User1                                          user1@gmail.com
       102 User2                                          user2@gmail.com
       103 User3                                          user3@gmail.com
       104 User4                                          user4@gmail.com

SQL> select *from Ratings;

   USER_ID   MOVIE_ID RATING_VALUE
---------- ---------- ------------
       101          1            5
       101          2            4
       101          3            5
       102          1            4
       102          4            5
       102          7            5
       103          2            5
       103          5            4
       103          6            4
       104          3            5
       104          4            4
       104          7            5

12 rows selected.
```

```
SQL> SELECT u.name, m.title, r.rating_value
  2  FROM users u
  3  JOIN ratings r ON u.user_id = r.user_id
  4  JOIN movies m ON m.movie_id = r.movie_id;

NAME                                           TITLE                                                                                  RATING_VALUE
---------------------------------------------- ---------------------------------------------------------------------------------------------
User1                                          Baahubali                                                                                     5
User1                                          RRR                                                                                           4
User1                                          Eega                                                                                          5
User2                                          Baahubali                                                                                     4
User2                                          Pushpa                                                                                        5
User2                                          Magadheera                                                                                    5
User3                                          RRR                                                                                           5
User3                                          Arjun Reddy                                                                                   4
User3                                          Srimanthudu                                                                                   4
User4                                          Eega                                                                                          5
User4                                          Pushpa                                                                                        4
User4                                          Magadheera                                                                                    5

12 rows selected.

SQL> SELECT m.title, AVG(r.rating_value) AS avg_rating
  2  FROM movies m
  3  JOIN ratings r ON m.movie_id = r.movie_id
  4  GROUP BY m.title
  5  ORDER BY avg_rating DESC;

TITLE                                                                                                  AVG_RATING
------------------------------------------------------------------------------------------------------ ----------
Eega                                                                                                            5
Magadheera                                                                                                      5
RRR                                                                                                           4.5
Pushpa                                                                                                        4.5
Baahubali                                                                                                     4.5
Srimanthudu                                                                                                     4
Arjun Reddy                                                                                                     4

7 rows selected.
```

```
SQL> SELECT m.title, r.rating_value
  2  FROM movies m
  3  JOIN ratings r ON m.movie_id = r.movie_id
  4  WHERE r.user_id = 101
  5  ORDER BY r.rating_value DESC;

TITLE                                                                                                RATING_VALUE
---------------------------------------------------------------------------------------------------- ------------
Baahubali                                                                                                       5
Eega                                                                                                            5
RRR                                                                                                             4

SQL> CREATE VIEW top_movies AS
  2  SELECT m.title, AVG(r.rating_value) AS avg_rating
  3  FROM movies m
  4  JOIN ratings r ON m.movie_id = r.movie_id
  5  GROUP BY m.title;

View created.

SQL> SELECT * FROM top_movies;

TITLE                                                                                                AVG_RATING
---------------------------------------------------------------------------------------------------- ----------
Baahubali                                                                                                   4.5
Arjun Reddy                                                                                                   4
Magadheera                                                                                                    5
Eega                                                                                                          5
Srimanthudu                                                                                                   4
RRR                                                                                                         4.5
Pushpa                                                                                                      4.5

7 rows selected.

SQL> |
```

The system successfully stores movie, user, and rating data in relational tables. Using SQL queries and join operations, the system generates movie recommendations based on user ratings.The results show that top-rated movies are identified accurately and displayed efficiently.

Interpretation

The results indicate that the database system effectively combines data from multiple tables to generate meaningful recommendations.Higher-rated movies are prioritized, showing that the recommendation logic works correctly.This demonstrates how relational databases can be used to analyze user preferences.

Implications

The system can be applied in real-world applications such as:

- Movie streaming platforms
- OTT services
- Online recommendation systems

It helps in:

- Providing personalized suggestions
- Improving user experience
- Reducing search time

5 Mapping Pos and SDGs

a. Program Outcomes(POs)Mapping

Table1:MappingofProgramOutcomes(POs)toProjectRelevance

PONo.	ProgramOutcome	Relevance
PO1	EngineeringKnowledge	Applied database concepts such as SQL, normalization, and relational models in developing the movie recommendation system.
PO2	ProblemAnalysis	Analyzed the problem of selecting relevant movies and designed a system to filter and recommend movies based on user ratings.
PO3	Design/DevelopmentofSolutions	Designed and developed a database system with tables, relationships, and queries to provide movie recommendations.
PO4	Conduct Investigations of Complex Problems	Investigated how user preferences and ratings affect recommendations using SQL queries and aggregation functions.
PO5	Engineering Tool Usage	Used Oracle SQL and PL/SQL tools to implement database operations and queries effectively.
PO8	Individual and Collaborative Team work	Worked in a team to design, implement, and test the database system.
PO9	Communication	Presented the project through reports, SQL outputs, and explanations clearly.
PO10	Project Management and Finance	Managed time and resources effectively to complete the project within the given schedule.
PO11	Life-Long Learning	Learned new concepts like joins, views, and database design which are useful for future development.

b.SustainableDevelopmentGoals(SDGs)Mapping

Table2:MappingofSustainableDevelopmentGoals(SDGs)toProjectRelevance

SDGNo.	Goal	Relevance
SDG4	Quality Education	Enhances learning by applying database concepts practically through project implementation.
SDG8	Decent Work and Economic Growth	Recommendation systems are widely used in industries like OTT platforms, contributing to digital economy growth.
SDG9	Industry, Innovation and Infrastructure	The project demonstrates innovation in data handling and recommendation systems using database technology.
SDG10	Reduced Inequalities	Provides equal access to movie recommendations for all users without bias.

6 Conclusion

The Movie Recommendation Database system was successfully designed and implemented using Oracle SQL. This project effectively demonstrates the application of relational database concepts such as table creation, normalization, primary and foreign key constraints, and SQL operations like joins, aggregation, and views. The system manages structured data related to movies, users, and ratings, and generates meaningful recommendations based on user preferences. By using SQL queries, the system efficiently identifies top-rated movies and provides personalized suggestions, thereby reducing manual effort in searching for suitable content. The implementation also ensures data integrity and consistency through proper constraint handling. Overall, the project highlights the importance of database systems in real-world applications such as recommendation engines used in digital platforms. It shows how data can be organized and processed to support decision-making and improve user experience.

Future Work / Recommendations

The system can be further enhanced by integrating advanced recommendation techniques such as collaborative filtering and machine learning algorithms to provide more accurate and dynamic suggestions. A graphical user interface (GUI) or web-based application can be developed to make the system more interactive and user-friendly. Additional features such as filtering based on language, actors, directors, and user reviews can be included to improve functionality.

Furthermore, the system can be connected to real-time databases or APIs to fetch updated movie information. Performance optimization techniques can be applied to handle large-scale datasets efficiently. Security features such as user authentication and data protection can also be implemented to make the system more robust and suitable for real-world deployment.

7 Bibliography

The following resources were referred to during the development and implementation of the Movie Recommendation Database system:

[1]Database System Concepts, Abraham Silberschatz, Henry F. Korth, S. Sudarshan, McGraw-Hill Education.
(Used for understanding core DBMS concepts such as normalization, keys, and relational models.)

[2]Oracle Database Documentation, Oracle Corporation.
Available at: https://docs.oracle.com/
(Used for SQL syntax, PL/SQL programs, and database implementation.)

[3]W3Schools – SQL Tutorial.
Available at: https://www.w3schools.com/sql/
(Used for basic and advanced SQL query references.)

[4]GeeksforGeeks – DBMS Section.
Available at: https://www.geeksforgeeks.org/dbms/
(Used for understanding joins, constraints, and real-world examples.)

Course materials, lecture notes, and lab manuals provided by the faculty of the Database Management Systems course.